

第一次作业

《白话机器学习算法》第1章读书笔记

这一章是本书入门总纲，让我对整个分析流程有了清晰的认识。想要做好一个数据挖掘项目，不是上来就套用算法，而是要经过四个严谨的步骤：准备数据，选择算法，参数调优，评价模型。

首先是准备数据。数据是所有工作的基础。工作中需要区分不同类型的变量，比如是简单的“是/否”，还是具体的数字。更重要的是能够处理比如数据缺失之类的问题，可以用出现最多的值去填，或者用模型去算，但最好不要直接忽略缺失部分，因为忽略数据可能会丢失重要的信息。

接着是选择算法。这一步让我明白了为什么要学那么多算法。如果想从数据里发现规律，例如给顾客分群类的工作，就要用无监督学习。如果我想根据已有的数据做预测，比如预测房价，就要用监督学习。还有一种更聪明的强化学习，它可以根据反馈边学边用，就像一个能自我优化的广告投放系统。

然后是参数调优。参数没调好，模型要么太敏感，把噪音都当成了信号，就是过拟合；要么太迟钝，连明显的规律都发现不了，就是欠拟合。找到一个适合使用的平衡点，模型才能有用又好用。

最后是评价模型。光说模型准确率90%是不够的，还得知道模型问题在哪。比如在地震预测里，漏报比虚报的后果严重得多，这时候就得用混淆矩阵这种工具，看清楚模型到底有什么问题。为了防止模型只在训练数据上表现好，还得用交叉验证的方法，把数据分几组轮流训练测试，这样得到的评价才靠谱。

感想：这一章就像一张地图，让我看到了数据科学的全貌。以前觉得机器学习是一门高深的学问，现在明白它其实是一套解决问题的完整流程。从处理数据到选择模型，再到优化和评估，每一步都有讲究。

《大数据挖掘》

第1-2章：什么是数据挖掘和怎么做

这两章讲清楚了数据挖掘的本质。它不只是“挖掘数据”，而是从海量的、有噪音的数据里，提取出事先不知道但又有价值的信息。书中把挖掘的内容分成了六大类，很清晰：关联（比如啤酒和尿布），叫回归（比如预测房价），分类（比如垃圾邮件），聚类（比如客户分群），预测，诊断（比如欺诈检测）。

书中给出了一个标准化的六阶段流程（DPEMED）。从定义目标开始，接着准备数据、探索数据、建立模型、评估模型，到最后部署上线。这让我意识到，数据挖掘是一个系统工程，不是一个算法就能搞定的事。书中强调，学习正确的方法论很重要，它能指导我们科学、规范地做项目，少走弯路。

第4-5章：准备数据和探索数据

这两章让我深刻体会到，数据挖掘70 % 的精力可能都花在正式建模之前。

数据准备这一步，收集数据只是开始，更重要的是做质量分析，要检查数据有没有缺失、错误。然后用预处理把数据洗干净，比如用均值填补空缺，或者对数据进行标准化，让不同尺度的指标可以放在一起比较。

数据探索这一步，我们可以创造一些新变量，比如把质量和体积组合成“密度”，这能更直接地反映分析对象的特征。数据可视化也是个强大的工具，一张散点图或箱线图，往往能比一堆数字更直观地揭示数据的分布和异常。如果变量太多，还要使用降维，比如用主成分分析（PCA），把很多相关的变量浓缩成几个最关键的代表，既简化了问题，又不丢失主要信息。

感想：读完这几章，最大的感受是数据挖掘的功夫很大一部分在数据本身。模型、算法、数据之间联系紧密，想要做好数据分析工作就要深刻理解三者之间的关系并加以熟练运用，未来在做一个项目时，绝对不能轻视这几项工作，它们往往是决定项目成败的关键。

大数据： 规模巨大、 无法传统工具处理的数据集。 核心4V： 数据量大、 类型多、 价值密度低、 速度快。

数据挖掘： 从大量数据中提取隐含、 未知但有价值的知识。 涵盖关联、 回归、 分类、 聚类、 预测、 诊断六大任务。

数据降维： 用更少变量表达原始主要信息。 PCA： 线性组合压缩相关变量， 保留差异最大方向。 t-SNE： 高维映射低维时保持相似点靠近， 用于可视化。

有监督/无监督： 有监督用带标签数据做预测（分类/回归）。 无监督用无标签数据探索规律（聚类/关联）。

数据结构： 结构化是规整二维表（Excel）。 半结构化有标记但不规整（XML、JSON）， 需解析。 非结构化无预定义结构（文本、图片）， 最难处理。

数据预处理与清洗： 预处理是建模前所有数据处理总称。 清洗是核心——填缺失、平滑噪声、剔除离群点、解决不一致。

标准化与归一化： 标准化 $(\text{值} - \text{均值}) / \text{标准差}$ ， 转为均值为0标准差1， 适合正态分布。 归一化 $(\text{值} - \text{最小值}) / (\text{最大值} - \text{最小值})$ ， 缩放到[0, 1]， 不限分布。

衍生变量： 通过组合变换原始变量创造新变量， 需业务理解。

第二次作业

1、数据回归方法的基本方法、步骤

数据回归方法主要用于描述、解释和预测变量之间的关系。基本步骤如下：

- 第一，明确研究问题，确定因变量 Y 和自变量 X 。
- 第二，收集并整理样本数据，检查缺失值、异常值和变量量纲。
- 第三，通过散点图、相关系数等方式观察变量之间是否存在相关关系。
- 第四，选择合适的回归模型，如一元线性回归、多元线性回归或 Logistic 回归。
- 第五，利用最小二乘法、极大似然估计或损失函数最小化等方法估计模型系数。
- 第六，使用 R^2 、F 检验、p 值、残差分析等指标评价模型效果。
- 第七，判断模型是否适合数据，并用于解释变量关系或预测新样本。

2、一元线性回归和多元线性回归的基本数学模型

一元线性回归模型:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

其中, y 是因变量, x 是自变量, β_0 是截距, β_1 是回归系数, ε 是随机误差。 β_1 表示 x 每增加一个单位时, y 的平均变化量。

多元线性回归模型:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

其中, x_1 到 x_p 表示多个自变量, β_1 到 β_p 表示各自变量对应的回归系数。 β_i 表示在其他变量不变的情况下, x_i 对 y 的影响程度。

矩阵形式可写为:

$$Y = X\beta + \varepsilon$$

3、 Logistic 回归的基本数学模型及含义

Logistic 回归主要用于因变量为 0/1 的二分类问题，例如是否贷款、股票涨跌、客户是否流失等。

设 $p = P(Y=1|X)$ ，表示样本属于 1 类的概率，则 Logistic 回归模型为：

$$p = 1 / [1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}]$$

也可以写成：

$$\ln[p/(1-p)] = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

其中， $p/(1-p)$ 称为发生比， $\ln[p/(1-p)]$ 称为对数发生比。Logistic 回归先根据自变量计算属于某一类的概率，再根据设定阈值将概率转化为 0 或 1。

4、 Logistic 回归与普通线性回归的联系与区别

联系:

二者都属于监督学习方法，都利用已有数据建立自变量和因变量之间的关系。
二者都需要估计模型系数，并可用于解释变量影响和预测新样本。
二者都可以处理多个自变量。

区别:

线性回归的因变量通常是连续数值，Logistic 回归的因变量通常是 0/1 类别。
线性回归直接预测具体数值，Logistic 回归预测的是属于某一类的概率。
线性回归输出结果没有固定范围，Logistic 回归通过 S 形函数把输出限制在 0 到1 之间。
线性回归常用最小二乘法估计参数，Logistic 回归常用极大似然估计或分类损失函数。
线性回归系数表示自变量对因变量数值变化的影响，Logistic 回归系数表示自变量对对数发生比的影响。

5、《白话机器学习算法》第六章 “回归” 读书笔记

第六章主要讲了回归分析的基本思想。回归分析的核心是寻找一条最佳拟合线，使尽可能多的数据点靠近这条线，从而用于预测。

书中用房价预测作为例子说明，单个变量可以形成简单趋势线，多个变量可以通过加权组合提高预测准确度。变量的权重就是回归系数，它表示某个预测变量相对于其他变量的重要程度。

回归系数可以帮助我们理解变量的影响方向和影响大小。若系数为正，说明变量与结果大体同向变化；若系数为负，说明变量与结果大体反向变化。经过标准化后，不同变量的系数更适合直接比较。

回归分析的优点是直观、计算快、容易解释，但也有局限性。例如，它对异常值比较敏感；如果自变量之间高度相关，可能出现多重共线性；如果真实关系不是线性的，普通线性回归效果可能不好。

我的理解是：回归分析不仅是为了预测结果，更重要的是理解变量之间的关系。但回归只能说明相关关系，不能直接证明因果关系。因此建模时不能只看 R^2 等指标，还要结合实际背景判断模型是否合理。

第三次作业

1、分类算法的基本逻辑和基本概念

分类算法的目标是根据已有数据学习出一个分类器，用来判断未知样本属于哪个类别。它属于监督学习，因为训练数据中已经包含类别标签。

分类的基本逻辑是：先用带有类别标记的历史数据训练模型，再用测试集检验模型效果，最后把训练好的模型用于新数据分类。

基本概念：

分类器：能够根据输入数据判断样本类别的模型。

训练集：用于训练模型的历史数据。

测试集：用于检验模型分类效果的数据。

类标签：样本已经标记好的类别，如“是/否”“涨/跌”。

开放集：需要使用模型进行分类的新数据。

准确率：预测正确的样本数量占总样本数量的比例。

混淆矩阵：用来展示分类结果正确与错误情况的表格，包括 TP、TN、FP、FN。

泛化能力：模型对新数据的预测能力，不能只在训练集上表现好。

2、KNN 算法的思想与步骤

KNN 即 K 近邻算法，基本思想是“物以类聚”。如果一个未知样本周围最近的 K 个样本大多数属于某一类，那么该未知样本也被判断为这一类。

KNN 算法步骤：

第一，确定训练集数据， 并给每个样本标注类别。

第二，选择距离度量方法，如欧氏距离。

第三，计算待分类样本与训练集中每个样本之间的距离。

第四，选出距离最近的 K 个训练样本。

第五，统计这 K 个邻居中各类别出现的次数。

第六，按照“少数服从多数”的原则，把待分类样本归入出现次数最多的类别。

KNN 的优点是原理简单、容易理解，不需要复杂训练过程。缺点是每次预测都要计算与大量样本的距离，计算量较大； K 值选择不合适会影响结果； 当类别不平衡或变量过多时，效果可能下降。

3、《白话机器学习》第七章 KNN 读书笔记

第七章主要介绍了 K 近邻算法。KNN 的核心思想是根据周围数据点的类别来判断新数据点的类别。例如判断红葡萄酒和白葡萄酒时，可以根据某种葡萄酒与已知样本在化学成分上的相似程度，推断它属于哪一类。

K 值是 KNN 中非常重要的参数。K 太小，模型容易受到噪声和异常点影响；K 太大，又可能忽略局部特征，使分类结果过于粗略。因此，K 值通常需要通过交叉验证等方法进行调优。

KNN 不仅可以用于分类，也可以用于异常检测。如果一个样本与周围样本差异很大，就可能是异常点。KNN 的优点是直观、简单，缺点是当预测变量过多、样本量很大或类别不平衡时，计算效率和准确性都会受到影响。

我的理解是，KNN 更像是一种基于经验的判断方法，它没有明显的训练模型，而是直接依赖历史样本之间的距离关系进行分类。

第四次作业

一，簇的概念

簇是聚类分析的基本输出单元，指一组具有相似特征的数据对象集合，核心遵循“类内相似、类间相异”的原则——同一簇内的数据相似度较高，不同簇间的数据相似度较低。

簇的定义依赖于数据的特性和期望的结果。簇的形象在空间分布上也不确定，可以拥有多种形状

簇的数量、边界及特征均由数据本身的分布决定（部分算法需人为指定簇数），是数据自然分组的体现，常见应用包括用户分群、图像分割等，其合理性可通过簇内平方和、轮廓系数等指标评估。

二， 类的两种度量方法

1、 欧氏距离

核心定义

欧氏距离是聚类分析中最基础、最常用的距离度量方法，用于衡量多维空间中两个数据对象（向量）之间的直线距离，核心反映数据在数值上的绝对差异，距离越小，两个数据对象的相似性越高，越适合归为同一簇。该方法对数值大小敏感，适用于低维、数值分布均匀的数值型数据。

计算公式

设两个 n 维向量分别为 $X=(x_1, x_2, \dots, x_n)$ 和 $Y=(y_1, y_2, \dots, y_n)$ ，则欧氏距离计算公式为：

$$d(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2、余弦距离

核心定义

余弦距离由余弦相似度推导而来，核心衡量两个数据对象（向量）在方向上的相似性，不关注向量的绝对数值大小（对数据尺度不敏感）。余弦相似度越接近 1，说明两个向量方向越一致，相似性越高；反之越接近 0，相似性越低。余弦距离是高维数据（如文本、特征向量）相似性度量的常用方法。

计算公式

余弦相似度计算公式： $\text{sim}(X, Y) = \cos\theta = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \cdot \sqrt{\sum_{i=1}^n y_i^2}}$

余弦距离计算公式（由余弦相似度转化）： $d(X, Y) = 1 - \text{sim}(X, Y)$

三， K - 均值 (K-Means) 聚类算法原理与步骤

K - 均值聚类是经典的划分聚类方法。给定包含 N 个元组的数据集，算法将其构造为 K 个分组（聚类），满足：每个分组至少含一条数据记录，每条记录属于且仅属于一个分组。算法通过迭代优化分组，目标是同一簇内数据尽可能相近，不同簇间数据尽可能远离，最终使簇的误差平方和达到最小。

工作原理：先随机选取 K 个点作为初始簇中心，计算各样本到中心的距离并分配至最近簇；再更新每个簇的均值中心，不断重复迭代。当簇划分不再变化、簇中心不再移动或误差平方和达到局部最小时，算法收敛。算法步骤

从 N 个数据对象中随机选择 K 个对象作为初始聚类中心。

根据各聚类中心，计算每个对象与中心的距离，按最小距离将对象重新划分到对应簇。

对每个簇，重新计算均值并更新聚类中心。

循环执行步骤 2、3，直到聚类不再发生变化，即无对象被重新分配、聚类中心不再改变，或误差平方和达到局部最小。

四， hc算法过程

1、凝聚式层次聚类（AGENES）

核心过程： 自底向上合并

初始化： 将每个样本单独作为一个簇。

找最近对： 计算所有簇间距离， 找到距离最近的两个样本点（代表两簇最近）。

合并簇： 将这两个簇合并为一个新簇。

迭代： 更新距离矩阵， 重复“找最近对 - 合并”。

终止： 簇数减至目标 K ， 停止并输出 K 个簇。

2、分裂式层次聚类（DIANA）

核心过程： 自顶向下拆分

初始化： 将所有样本归并为一个簇。

找最远对： 在当前簇内，找到距离最远的两个样本点（作为拆分代表）。

拆分簇： 以这两点为中心，将原簇中所有样本按距离远近分配到两个新簇。

迭代： 重复“找最远对 - 拆分”，每次迭代簇数增加。

终止： 簇数增至目标 K ，停止并输出 K 个簇。

五，

1， 基础结构

树状结构完整呈现了样本从单个独立簇逐步合并为大簇的全过程， 直观展示了样本间的亲疏关系： 纵轴高度越高的簇， 样本间差异越大； 纵轴高度越低的簇， 样本相似度越高。

横轴为数据点编号， 通过树的分支可清晰追溯每个样本的聚类路径， 明确任意两个样本 / 簇的合并顺序与归属关系。

纵轴为样本间标准距离， 横线的高度直接对应两个簇合并时的距离值： 高度越高， 代表合并的两个簇差异越大； 高度越低， 代表簇内样本越相似。

2，核心信息

通过改变水平截断线高度，可灵活改变最终聚类结果：

截断线与树结构的交点数量 = 最终簇数（示例中红线相交 3 次，对应 3 个簇）

截断线位置可按需调整：向下平移（距离更小）→ 簇数增多、分类更细；向上平移（距离更大）→ 簇数减少、分类更粗

聚类的紧凑性与分离度

簇内分支高度低：说明簇内样本相似度高、聚类紧凑

簇间合并高度差大：说明不同簇之间差异明显、分离度好，聚类效果佳

六,

本章介绍的 K 均值聚类 (K-Means) *是一种典型的划分式聚类算法，主要用于将相似的数据点自动归为不同群组，常应用在顾客分群、用户偏好分析、精准营销等实际场景中。在分析之前，通常并不知道应该分成多少组、每组包含哪些对象，而 K mean聚类正是用来解决这类问题的常用方法。

使用 K mean聚类需要明确两个核心问题：确定合适的群组数量 K，以及确定每个群组包含哪些数据。群组数量不能过多或过少，组数太多会导致组间差异不明显，太少又无法提取有效模式。实际中可通过陡坡图辅助判断，曲线明显拐弯的位置（肘部）一般对应较优的 K 值。

确定群组成员依靠迭代过程完成：先随机选取 K 个伪中心点，将每个数据点分配到最近的中心点所属群组；再根据当前群组内的数据分布更新中心点位置，重复执行分配和更新步骤，直到群组不再发生变化，算法达到收敛。该方法不仅适用于二维数据，也可扩展到更高维度的特征分析。

K 均值聚类简单直观、易于实现，但也存在明显局限性：它要求每个数据点仅属于一个群组，默认群组呈圆形、离散且互不重叠，因此在处理非圆形、重叠或嵌套结构的数据时效果较差。实际应用中，常先用 K 均值做初步探索，再结合更复杂的聚类方法进行深入分析。

第五次作业

1、关联选股的背景知识

行业关联选股法是把关联规则算法应用到股票市场分析中，用来发现行业或股票之间的联动关系。

股票市场中，不同行业之间往往存在产业链关系、政策影响关系、资金流关系和市场情绪关系。例如，新能源汽车行业上涨时，锂电池、充电桩、稀有金属等行业可能也会联动上涨；房地产行业上涨时，建材、家电、银行等行业也可能受到影响。

关联选股的基本思想是：把每个交易日看作一个事务，把行业或股票看作项，把当天上涨或满足条件的行业记为 1，不满足条件的记为 0。然后使用 Apriori 算法找出经常共同出现的行业组合，并进一步分析行业之间的关联规则。

这种方法可以辅助发现行业联动关系，为行业轮动、板块分析和选股提供参考。但它只能说明统计关联，不能直接证明因果关系。

2、数据分析：矩阵行与列的含义

第五讲文件夹中的对应数据文件是一个典型的 0-1 矩阵示例。表中行是事务，列是项，元素是 0 或 1。

在行业关联选股中也可以使用同样的矩阵结构：

行表示交易日。每一行代表一天的市场情况，也就是一个事务。

列表示行业或股票。每一列代表一个行业或一只股票。

矩阵元素表示该行业或股票当天是否满足条件。

例如：

1 表示该行业当天上涨，或涨幅超过设定标准。

0 表示该行业当天没有上涨，或没有达到设定标准。

如果某一天新能源汽车、锂电池、充电桩三个行业都上涨，那么这一行中这三个行业对应的值为 1，其他未上涨行业对应的值为 0。

这种矩阵的意义是把现实中的交易数据转化为 Apriori 算法可以处理的事务数据。

3、运行程序

运行文件夹中的 Python 程序。虽然它使用的是商品数据，但程序逻辑可以直接迁移到行业关联选股中。

程序主要步骤如下：

第一，导入相关库，包括 pandas、TransactionEncoder、apriori、networkx 和 matplotlib。

第二，读取 products.csv 数据。

第三，把每一行中的“商品1、商品2、商品3”转化为一个事务列表。

第四，使用 TransactionEncoder 把事务数据转化为 0-1 矩阵。

第五，调用 Apriori 算法计算频繁项集。程序中设置的最小支持度是 0.02，表示某个组合至少出现 2 次。

第六，只保留 2 项集，并统计共现次数。

第七，使用 networkx 构建商品关联网络图。

第八，把结果保存到 product_associations.csv。

如果应用到行业关联选股中，只需要把程序中的“商品”替换为“行业”：

products.csv 可替换为行业涨跌数据。

“商品1、商品2、商品3”可替换为某交易日上涨的行业列表。

每一行商品组合可理解为某一天共同上涨的行业组合。

Apriori 输出的频繁项集可理解为经常共同上涨的行业组合。

知识图谱中的节点表示行业，边表示两个行业之间存在共现关系，边的权重表示共同出现次数。

4、程序结果分析与思考

程序运行后会生成关联结果，记录了商品之间的共现关系。共现次数越高，说明两个商品在同一事务中同时出现的次数越多，关联程度相对更明显。

例如结果中“无线鼠标和硬盘”共现 5 次，“电动牙刷和笔记本电脑”共现 5 次，“笔记本电脑和路由器”共现 4 次。这说明在当前样本数据中，这些商品组合出现得比较频繁，可以认为它们之间存在一定关联。

如果把这个过程应用到行业关联选股中，就可以把商品替换成行业，把一次购物记录替换成一个交易日。若某两个行业经常在同一天上涨，就说明它们可能存在联动关系。例如新能源汽车和锂电池经常共同上涨，可能是因为它们处在同一产业链中，容易受到相同政策、资金或市场情绪影响。

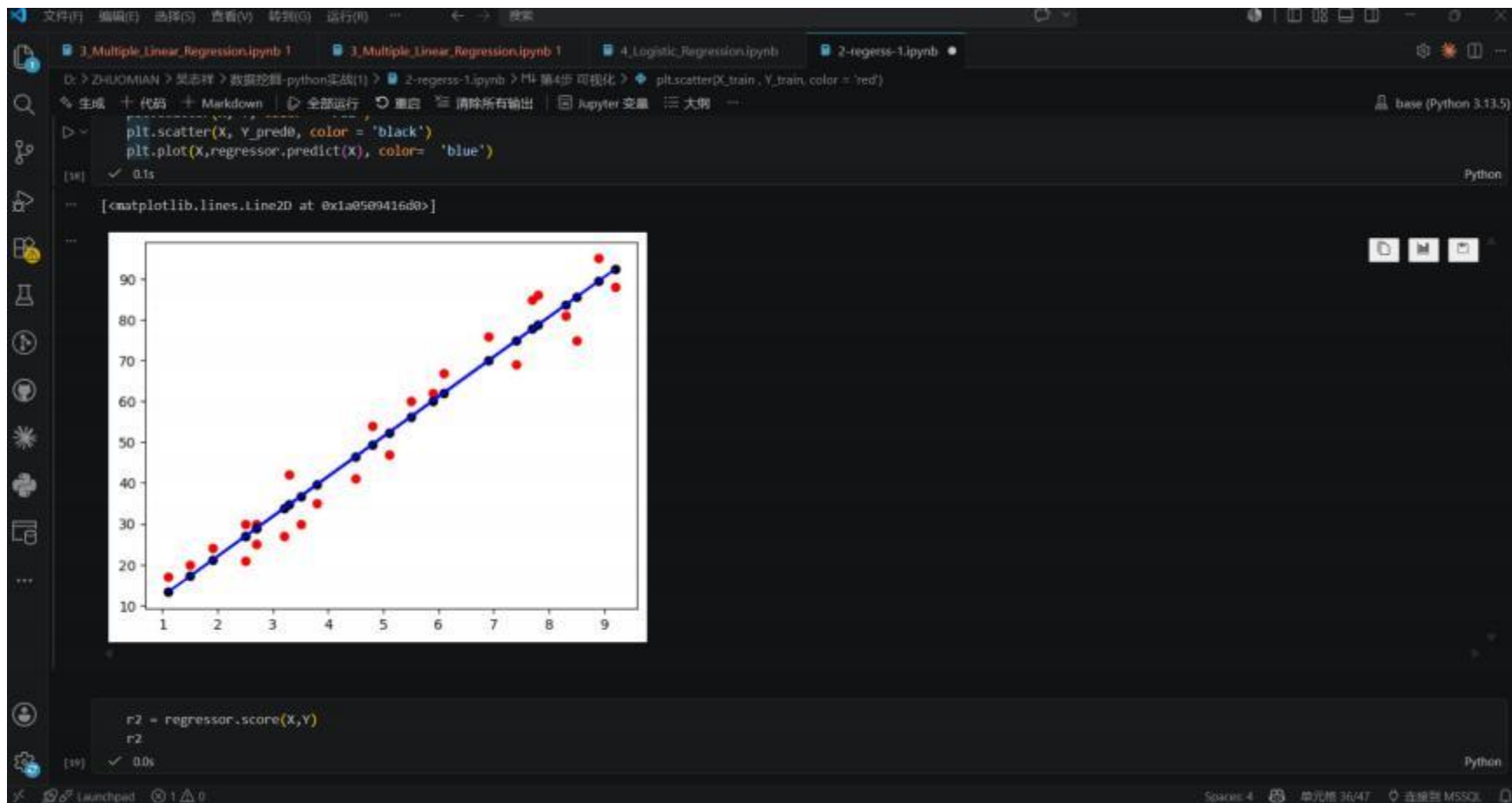
但需要注意，关联规则只能说明“经常一起出现”，不能直接说明因果关系。共现次数高不一定代表投资价值高，还要结合支持度、置信度、提升度以及行业背景进行判断。尤其是提升度，如果大于 1，说明这种关联比随机情况更强；如果接近 1，说明关联可能并不明显。

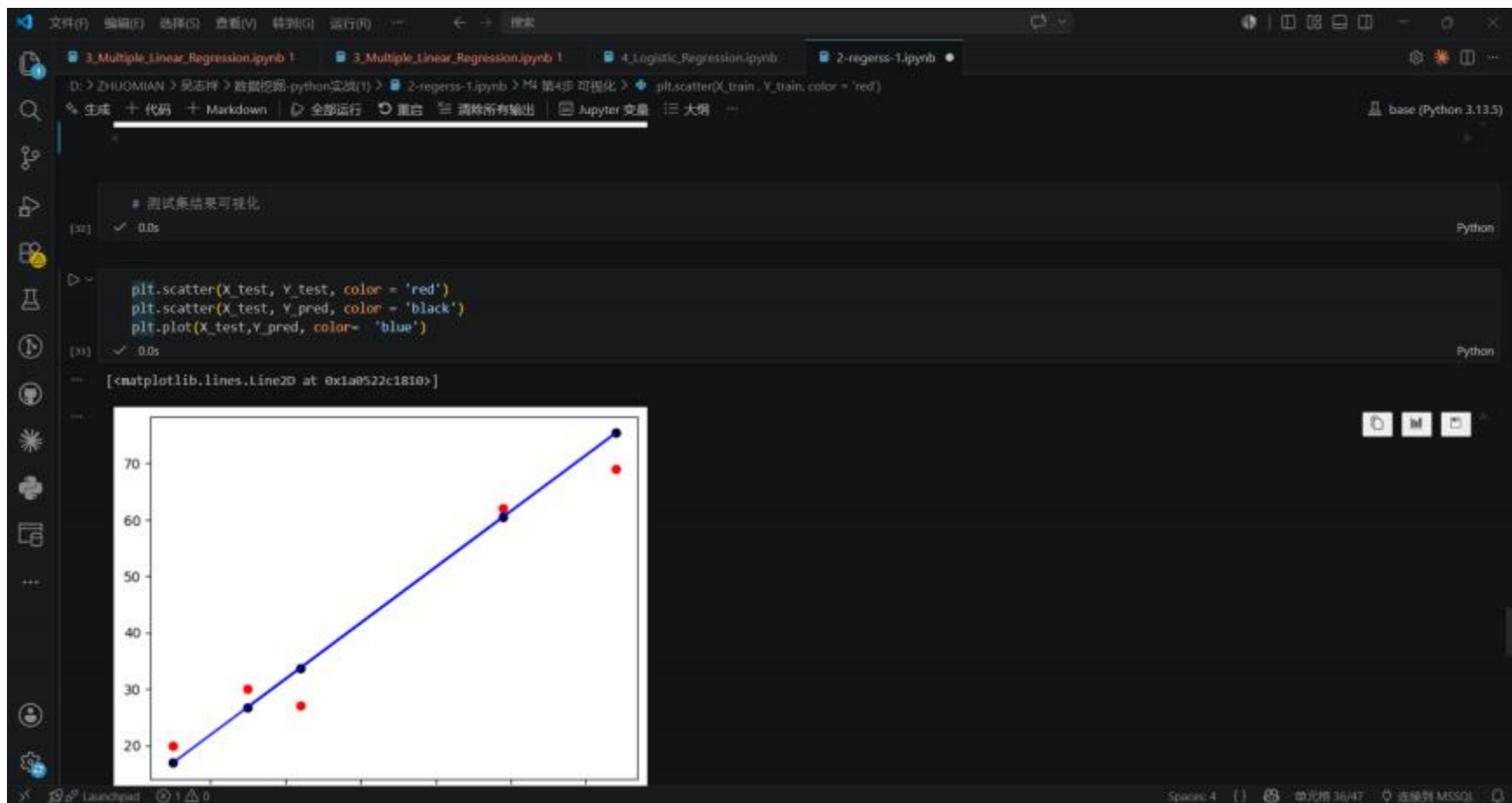
因此，行业关联选股法可以作为辅助分析工具，用来发现行业之间的联动关系，但不能单独作为买卖股票的依据。

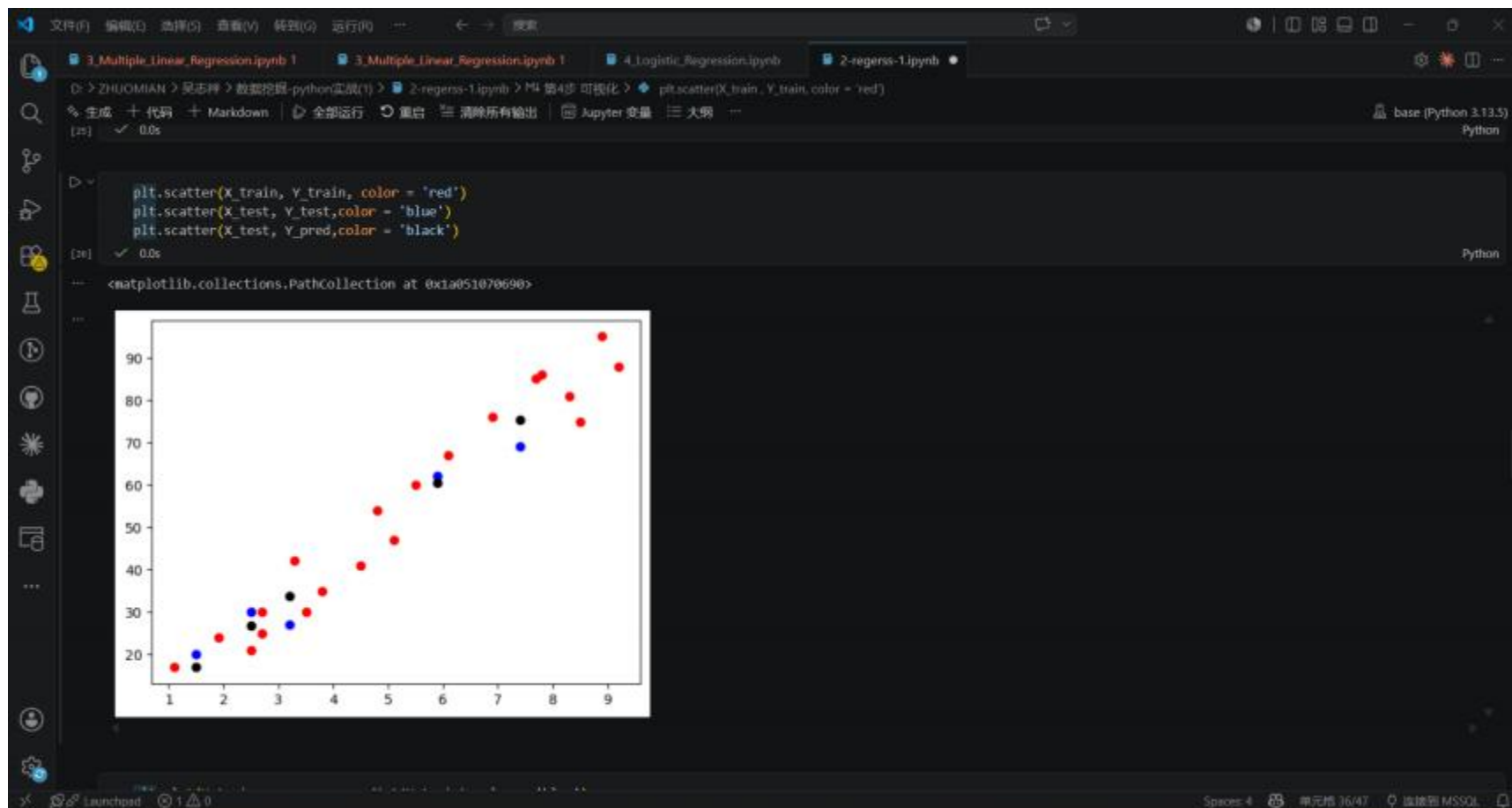
第六次作业

一元 线性回归

代码运行结果







一、为什么 x 必须是列向量？变成行向量会怎么样？

因为sklearn里要求输入的特征 x 必须是二维的，行代表样本，列代表特征。哪怕只有一个特征，也要写成 n 行1列的形式，也就是列向量。

如果写成行向量，模型会认为只有一个样本但有多个特征，这样样本数和标签就对不上，会直接报错。

二、如何显示生成的“熟模型”？ $y = b_0 + b_1 * x$, b_0 和 b_1 怎么获得？

训练完模型后， b_1 存在`regressor.coef_`里， b_0 存在`regressor.intercept_`里。

- 跑出来的结果是 b_1 约等于9.91， b_0 约等于2.02， 所以模型就是：
分数 = 2.02 + 9.91乘以hours。
-

三、怎么评价模型好坏？ R方是多少

用score()方法就能得到R方， R方越接近1说明拟合越好。训练集的R方约0.9516，测试集的R方约0.9455，两个都很高而且差不多，说明模型拟合得不错，也没有明显过拟合。

- 3 评价结果的好坏 R^2

```
R2_train = regressor.score(X_train,Y_train)
R2_train
```

✓ 0.0s

0.9515510725211552

```
R2_test = regressor.score(X_test,Y_test)
R2_test
```

✓ 0.0s

0.9454906892105356

四、调整训练/测试的比例，对结果 (R^2) 有什么影响？试试看

第一，如果增大测试集比例，比如改成0.3或0.4，意味着训练集变小了。训练集变小，模型学到的东西就少了，训练集上的 R 方通常会下降。但测试集变大了，测试集上的 R 方会更稳定、更有代表性，不容易因为运气好或差而波动太大。

第二，如果减小测试集比例，比如改成0.1，训练集更大了，模型能学到更多信息，训练集 R 方可能会更高。但测试集太小，测试结果就不太靠谱，可能这次测出来很高，下次换一批测试样本又很低，波动会比较大。

第三，如果把测试集比例设得特别大，比如0.5，训练集只剩下一半数据，模型可能学不到完整的规律，训练集和测试集的 R 方都会明显下降

原比例:

```
• 3 评价结果的好坏  $R^2$ 

R2_train = regressor.score(X_train,Y_train)
R2_train
✓ 0.0s
0.9515510725211552

R2_test = regressor.score(X_test,Y_test)
R2_test
✓ 0.0s
0.9454906892105356
```

比例3:7

```
• 3 评价结果的好坏  $R^2$ 

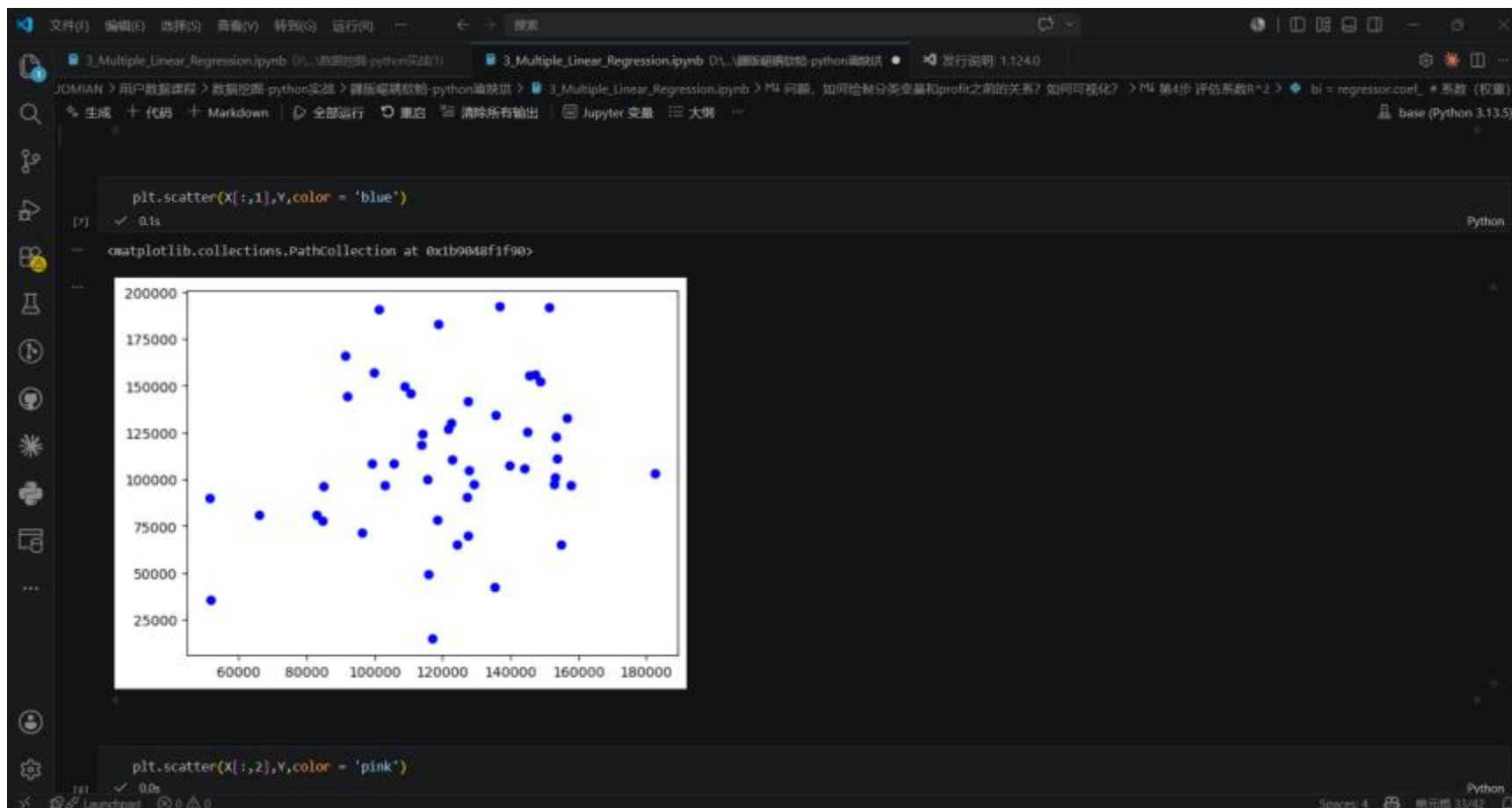
R2_train = regressor.score(X_train,Y_train)
R2_train
✓ 0.0s
0.9484997422695115

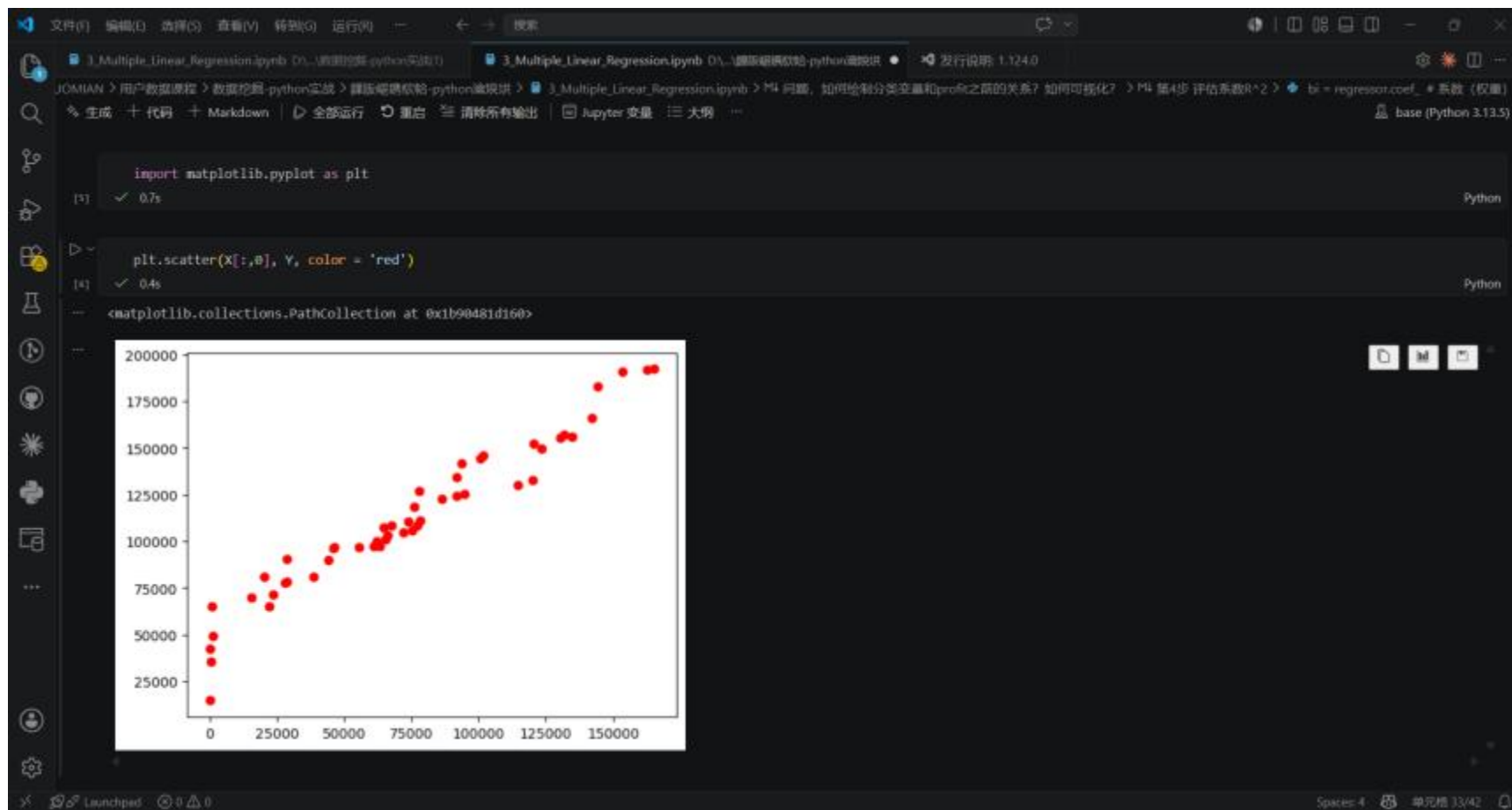
R2_test = regressor.score(X_test,Y_test)
R2_test
✓ 0.0s
0.9568211104435257
```

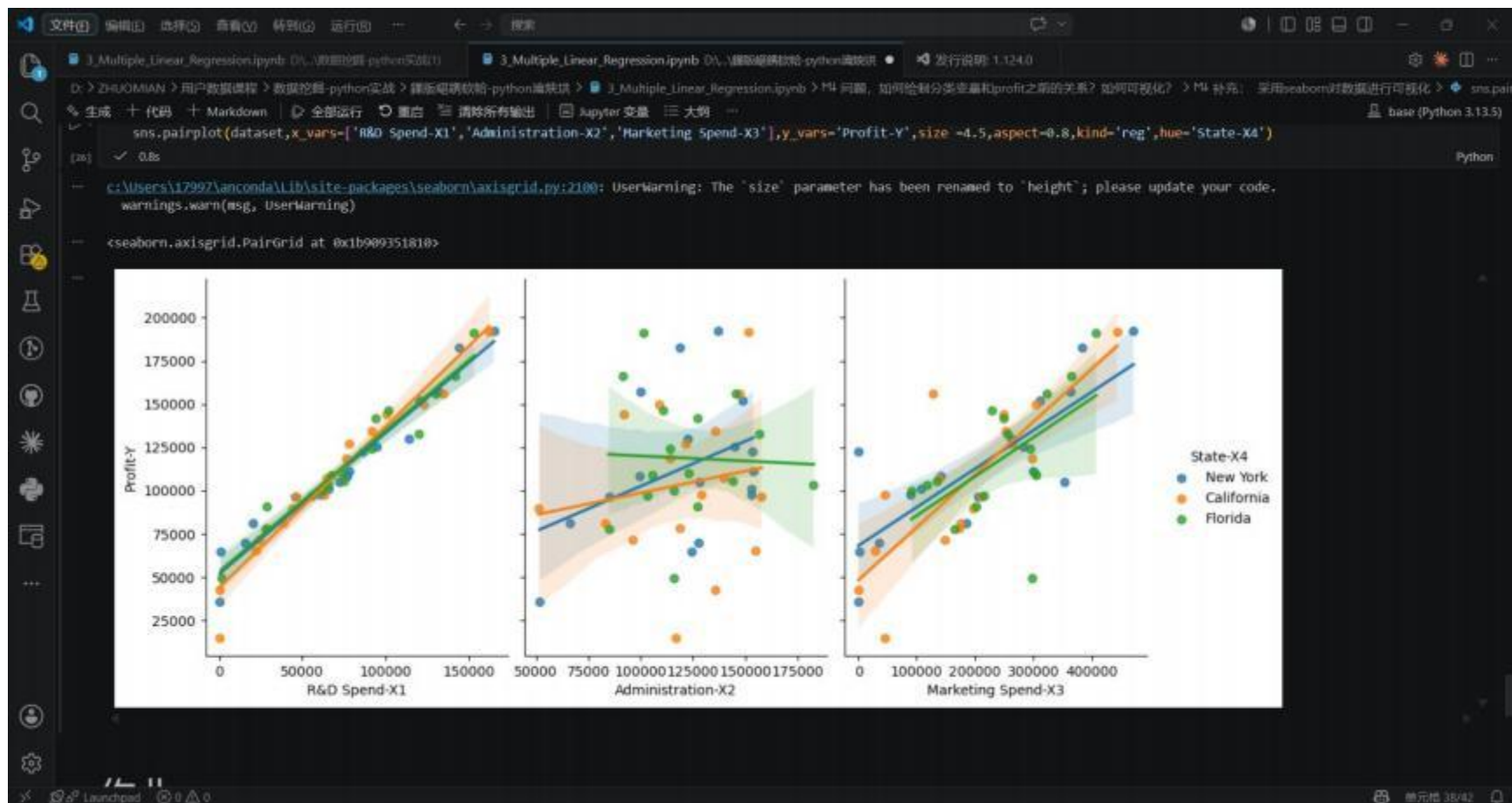
第七次作业

多元线性回归

代码运行结果







1 观察数据集的特征，有几个属性？属性中的分类变量是什么？可视化分类变量（城市）和因变量（营收）之间的关系。

数据集有5个属性： R&D Spend、Administration、Marketing Spend、State、Profit。其中State是分类变量，有三个取值： New York、California、Florida。从可视化结果来看，不同城市的利润分布有差异，Florida的平均利润最高， New York次之， California最低。

2 如何把分类变量转化为onehot, 过程是什么? 有了上一次的经验, 这一次你可以自己默写代码了嘛?

转成one-hot的步骤是: 先用LabelEncoder把文字标签 (New York、California、Florida) 转成数字0、1、2; 然后用OneHotEncoder把这列数字转成二进制的哑变量, 有几个类别就生成几列, 每列只有0和1。

3 如何评估结果? R^2

评估模型可以用 R^2 ,本案例结果 R^2 约0.95,说明模型能解释大约95%的数据变异,拟合效果很好。同时MSE、MAE、RMSE这些指标越小越好。



```
第4步 评估系数 $R^2$   
  
R2 = regressor.score(X,Y)  
R2  
✓ 0.0s  
0.9490884301964866
```

The image shows a Jupyter Notebook interface. At the top, the text '第4步 评估系数 R^2 ' (Step 4: Evaluate coefficient R^2) is displayed. Below it, a code cell contains the line 'R2 = regressor.score(X,Y)'. The output of this cell is 'R2', followed by a green checkmark and '0.0s' indicating execution time, and finally the numerical value '0.9490884301964866'.

4 如何获得系数（属性的权重），以及截距？怎么分析这个结果？最终的数学公式应该是什么？

系数存在regressor.coef_里，截距存在regressor.intercept_里。系数的正负表示该变量对利润的影响方向，绝对值越大影响越明显。最终公式是： $\text{Profit} = b_0 + b_1 \times \text{R\&D} + b_2 \times \text{Administration} + b_3 \times \text{Marketing} + b_4 \times \text{State_California} + b_5 \times \text{State_Florida}$ ，其中New York作为基准城市，它的效果体现在截距里。

5 再次思考本次案例采用数据的特征， 分类变量的存在， 对多元线性回归带来怎样的影响

分类变量的存在， 意味着不同城市的基础利润水平不同， 如果不处理直接当成数值0、 1、 2， 模型会错误地认为城市之间有序号大小关系， 这是不合理的。 所以必须用one-hot转换成哑变量。

6 能否把分类变量直接lable化，也就是变成0、1、2后直接计算？不转化称one-hot合理吗？

- 不能直接把分类变量label成0、1、2后直接计算。因为这样会引入虚假的顺序关系，比如模型会认为California（0）小于Florida（1）小于New York（2），但实际上城市之间没有这种大小关系。所以需要用one-hot转换。

KNN算法学习作业

1, 加入性别变量并进行onehot编码

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

加入性别变量后的数据结果

加入性别变量后的混淆矩阵输出样式

```
x = dataset.iloc[:,1:4].values  
y = dataset.iloc[:,4].values  
x[0:5]
```

✓ 0.0s

```
array([[ 'Male', 19, 19000],  
       [ 'Male', 35, 20000],  
       [ 'Female', 26, 43000],  
       [ 'Female', 27, 57000],  
       [ 'Male', 19, 76000]], dtype=object)
```

可以看到输出中多了性别列，表示其纳入了计算变量中

2.1比较KNN与Logistic在这个数据案例上的结果差异

```
# Logistic 回归对比
log_reg = LogisticRegression(random_state=0)
log_reg.fit(X_train, y_train)
y_pred_log = log_reg.predict(X_test)
acc_log = accuracy_score(y_test, y_pred_log)
print('Logistic Regression Accuracy:', acc_log)

# 混淆矩阵
print('KNN 混淆矩阵:', confusion_matrix(y_test, y_pred))
print('Logistic 混淆矩阵:', confusion_matrix(y_test, y_pred_log))
```

✓ 0.0s

Logistic Regression Accuracy: 0.9

KNN 混淆矩阵: [[64 4]

[3 29]]

Logistic 混淆矩阵: [[65 3]

[7 25]]

```
(variable) knn: KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
acc_knn = accuracy_score(y_test, y_pred)
print('KNN Accuracy:', acc_knn)
```

✓ 0.0s

KNN Accuracy: 0.93

KNN 在预测购买人群（标签 1）时更准确，假负较少（3 对 7）。Logistic 回归在预测未购买人群（标签 0）稍好，假正较少（3 对 4）。原因可能是 KNN 更适合捕捉数据中的非线性模式，而 Logistic 回归假设边界线性，对非线性边界的购买人预测稍差。

总得来看，针对这个数据集knn算法有更高的整体准确率且能更好的识别购买人群，更适合这个数据集

2.2 训练集的比例，如何影响结果

```
# 训练集比例实验
for test_size in [0.2, 0.25, 0.3, 0.4]:
    X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X, y, test_size=test_size, random_state=0)
    X_train_r = sc.fit_transform(X_train_r)
    X_test_r = sc.transform(X_test_r)
    knn.fit(X_train_r, y_train_r)
    y_pred_r = knn.predict(X_test_r)
    print(f'Test size={test_size}, KNN Accuracy={accuracy_score(y_test_r, y_pred_r)}')
```

✓ 0.0s

```
Test size=0.2, KNN Accuracy=0.95
Test size=0.25, KNN Accuracy=0.93
Test size=0.3, KNN Accuracy=0.9166666666666666
Test size=0.4, KNN Accuracy=0.90625
```

训练集比例直接影响模型的学习能力和测试准确率。

训练集比例大（测试集比例小）：模型拥有更多样本来学习数据模式，KNN 的邻居投票更可靠，训练和测试准确率通常较高，但测试集样本少，准确率波动可能增大。

训练集比例小（测试集比例大）：训练样本不足，模型学习到的数据模式有限，KNN 可能无法正确识别某些类别，测试准确率下降，容易出现欠拟合。

平衡选择：通常测试集比例 0.25–0.3 是经验值，既保证训练数据充足，又有足够测试样本评估模型性能。

总结来说，训练集越大，模型越“强”，但测试集太小会影响准确率的可靠性；训练集太小，则模型学习不足，预测效果差。

2.3超参数K如何影响结果

```
# K 值实验
for k in [1, 3, 7, 9]:
    knn_k = KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(X_train, y_train)
    y_pred_k = knn_k.predict(X_test)
    print(f'K={k}, Accuracy={accuracy_score(y_test, y_pred_k)}')
```

✓ 0.0s

K=1, Accuracy=0.92
K=3, Accuracy=0.92
K=7, Accuracy=0.93
K=9, Accuracy=0.93

超参数 K（邻居数）对 KNN 的结果有显著影响，主要体现在 偏差-方差平衡 上：

1. K 值较小（如 K=1）

模型只参考最近的一个邻居来预测。

优点：训练集拟合非常精确，训练准确率接近 100%。

缺点：对噪声敏感，测试集容易预测错误 → 过拟合。

2. K 值适中（如 K=3, 5, 7）

模型考虑多个邻居投票，既参考局部模式，又不过度拟合噪声。

优点：训练和测试准确率都比较平衡 → 最佳选择。

对数据的非线性模式有较好捕捉能力。

3. K 值较大（如 K=9 或更高）

模型参考的邻居更多，投票覆盖整个局部甚至全局。

优点：噪声影响小，模型更平滑。

缺点：决策边界过于平滑，可能无法捕捉局部非线性 → 欠拟合，训练和测试准确率下降。

总结：

K 太小：低偏差、高方差 → 训练集拟合好，但测试集易出错（过拟合）

K 适中：偏差和方差平衡 → 最佳性能

K 太大：高偏差、低方差 → 无法捕捉局部特征（欠拟合）

2.4 什么情况下会出现过拟合

过拟合通常发生在模型过于“贴合训练数据”而无法泛化到新样本的情况下，在KNN算法里面，常见的引发过拟合的原因有以下几点：

1, K 值太小 (如 $K=1$)

模型只参考最近的一个邻居预测，训练集几乎全部正确。

测试集上，少数噪点会导致预测错误 → 典型过拟合。

2, 特征维度过多

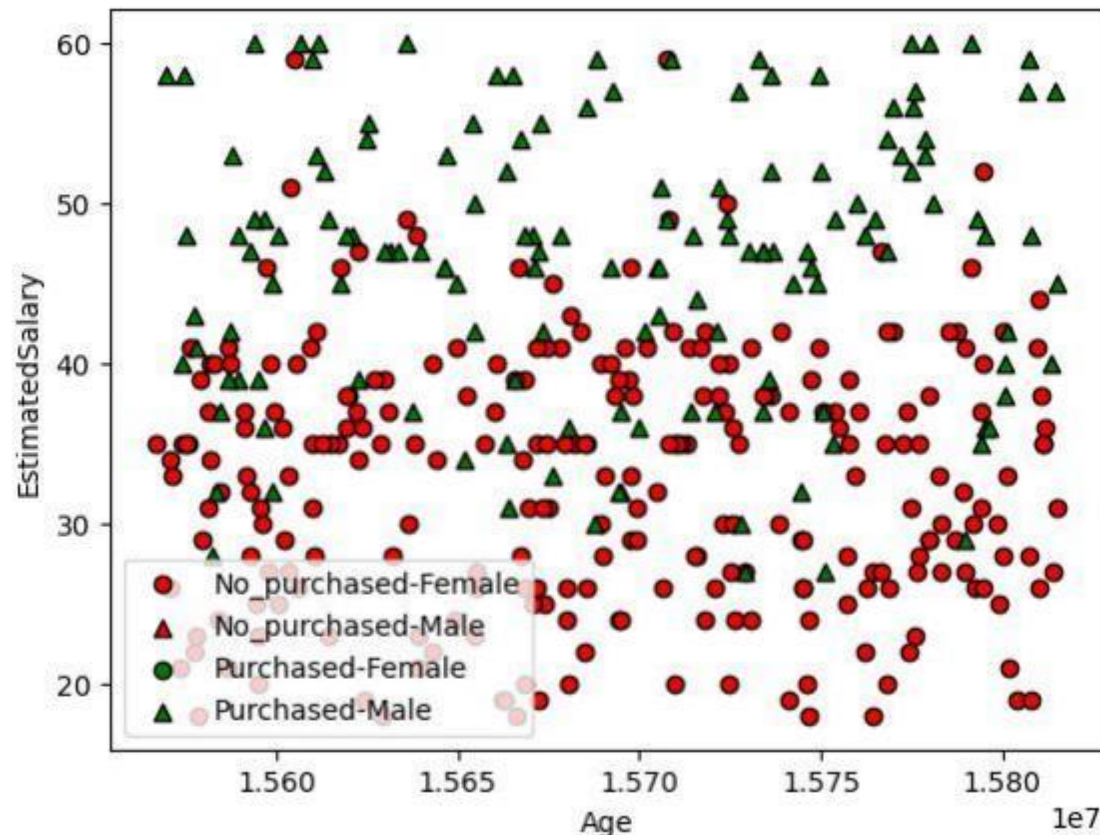
当特征数量大于样本量时，模型可能“记住”训练集的每个点。

即使某些特征与目标关系弱，也会被模型考虑进决策 → 模型在训练集表现好，但测试集效果差。

3, 数据量不足或噪声过多

样本数量少、噪声多时，模型容易捕捉到随机波动，而不是数据趋势的真实情况。

3, 对可视化图进行修改



图表介绍:

横轴和纵轴

横轴: Age (年龄)

纵轴: EstimatedSalary (估算薪资)

颜色表示购买状态

红色: 未购买 (No purchased)

绿色: 已购买 (Purchased)

形状表示性别

圆形: 女性

三角形: 男性

总结:

购买用户 (绿色点) 在年龄上并没有特别集中在某个段, 说明年龄对购买行为的影响可能不是特别强。但30岁以下的区域绿点数量相对较少, 可能与产品受众有关。

明显可以看到: 高薪资用户更可能购买广告 (绿色点集中在上方), 低薪资用户大多未购买 (红色点集中在下方)。说明薪资是影响购买行为的重要特征。

点形状区分性别: 圆形为女性, 三角形为男性。

从分布来看, 男性和女性在购买行为上没有显著差异:

高薪用户中, 男女都有购买者

低薪用户中, 男女都有未购买者

性别在整体趋势上影响较小, 但可以用作辅助特征 (微调 KNN 的邻居计算)

第九次作业

决策树

1. 加入性别因素，看看准确率的变化

添加前

可能由于计算的化简原因没有变化

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
accuracy
[44] ✓ 0.0s
... np.float64(0.925)
```

添加后

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
accuracy
] ✓ 0.0s
np.float64(0.925)
```

2. 如果不进行特征缩放，预测准确性有什么变化

决策树和随机森林是基于阈值的模型，不依赖距离计算，所以不需要特征缩放。你不做标准化，直接拿原始数据（年龄从十几到几十，薪资从几千到十几万）去训练，准确率基本不会有变化。。

3. 比较KNN、逻辑回归、决策树在该数据集上的表现

```
=====
模型比较
=====
```

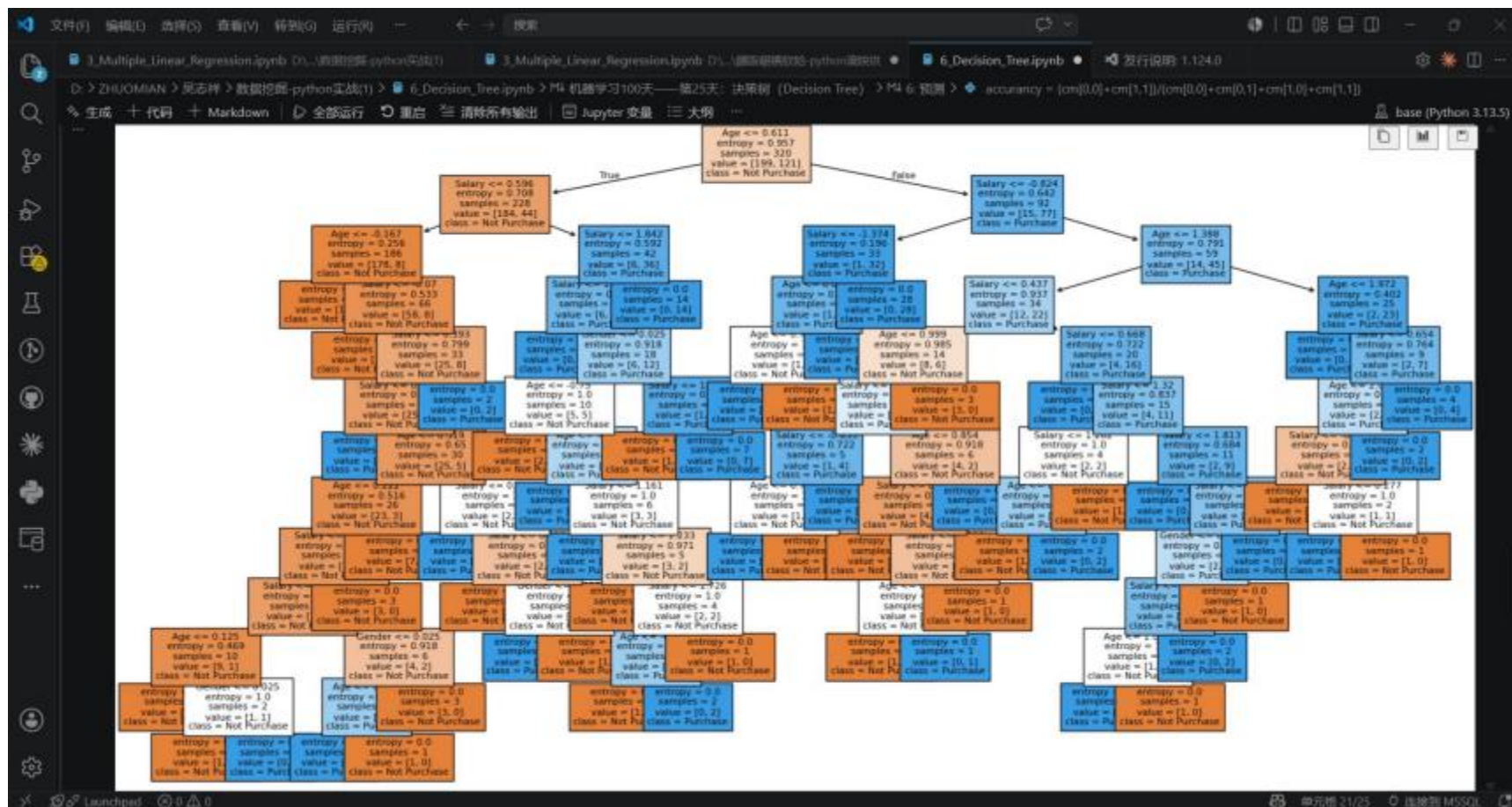
```
随机森林准确率: 0.9250
```

```
决策树(单棵)准确率: 0.9000
```

```
逻辑回归准确率: 0.9125
```

```
KNN准确率: 0.9500
```

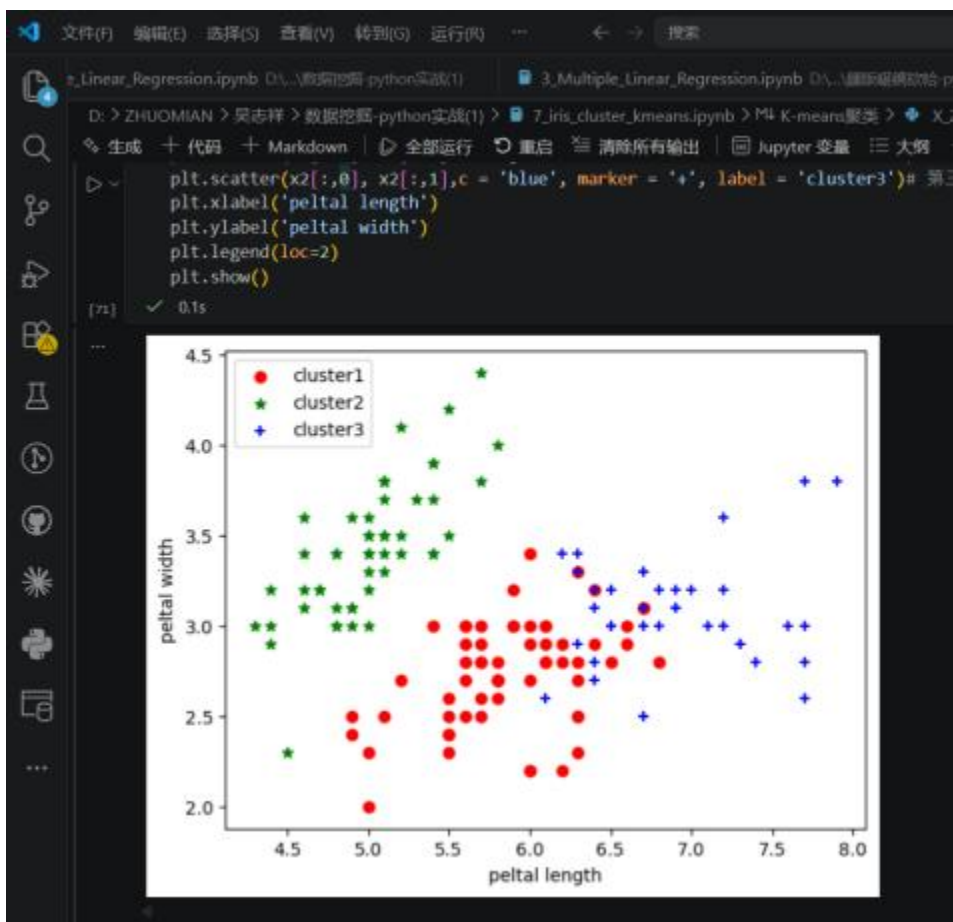

4. 把决策树输出看看长什么样



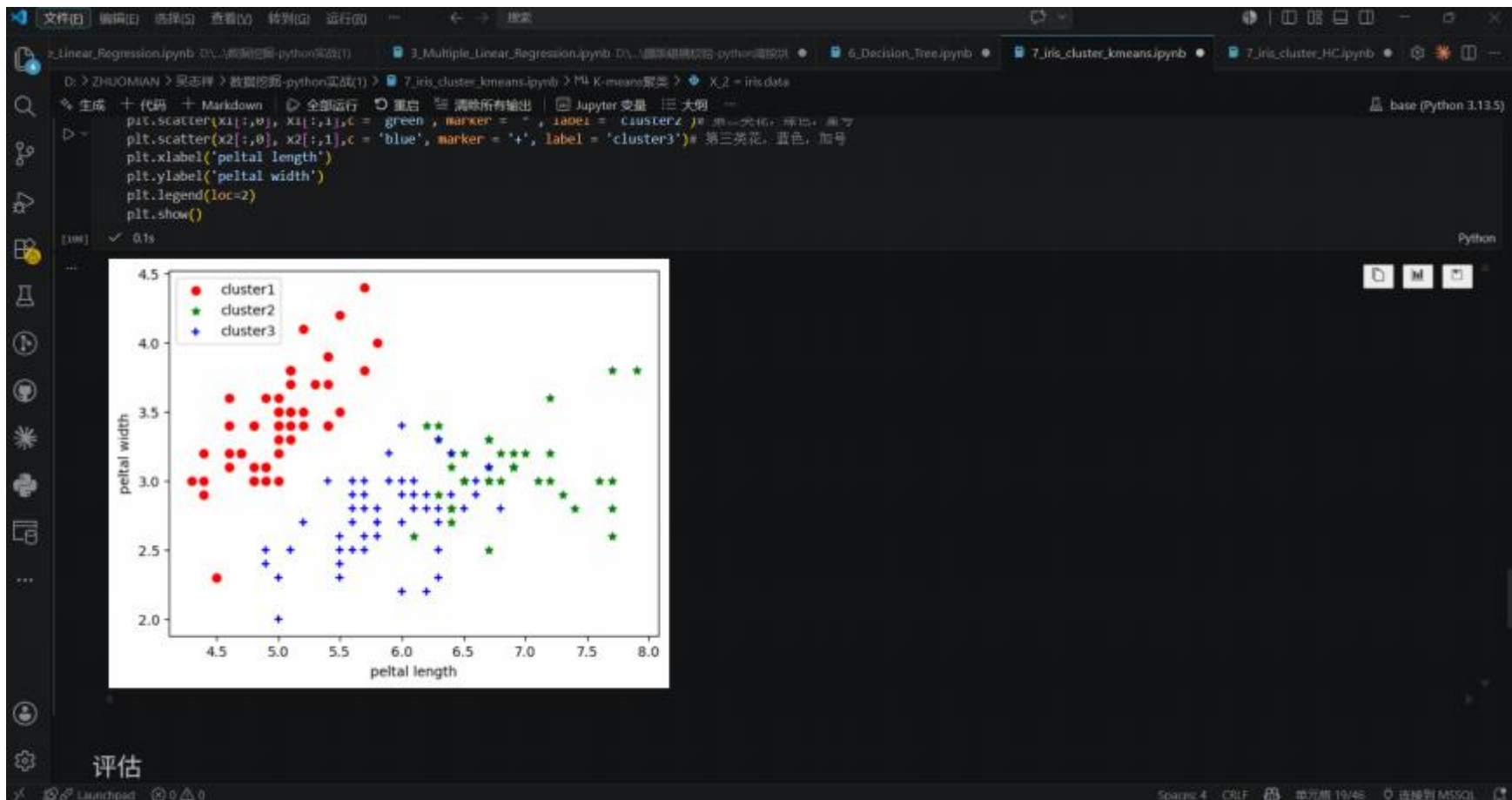
第十次作业

聚类

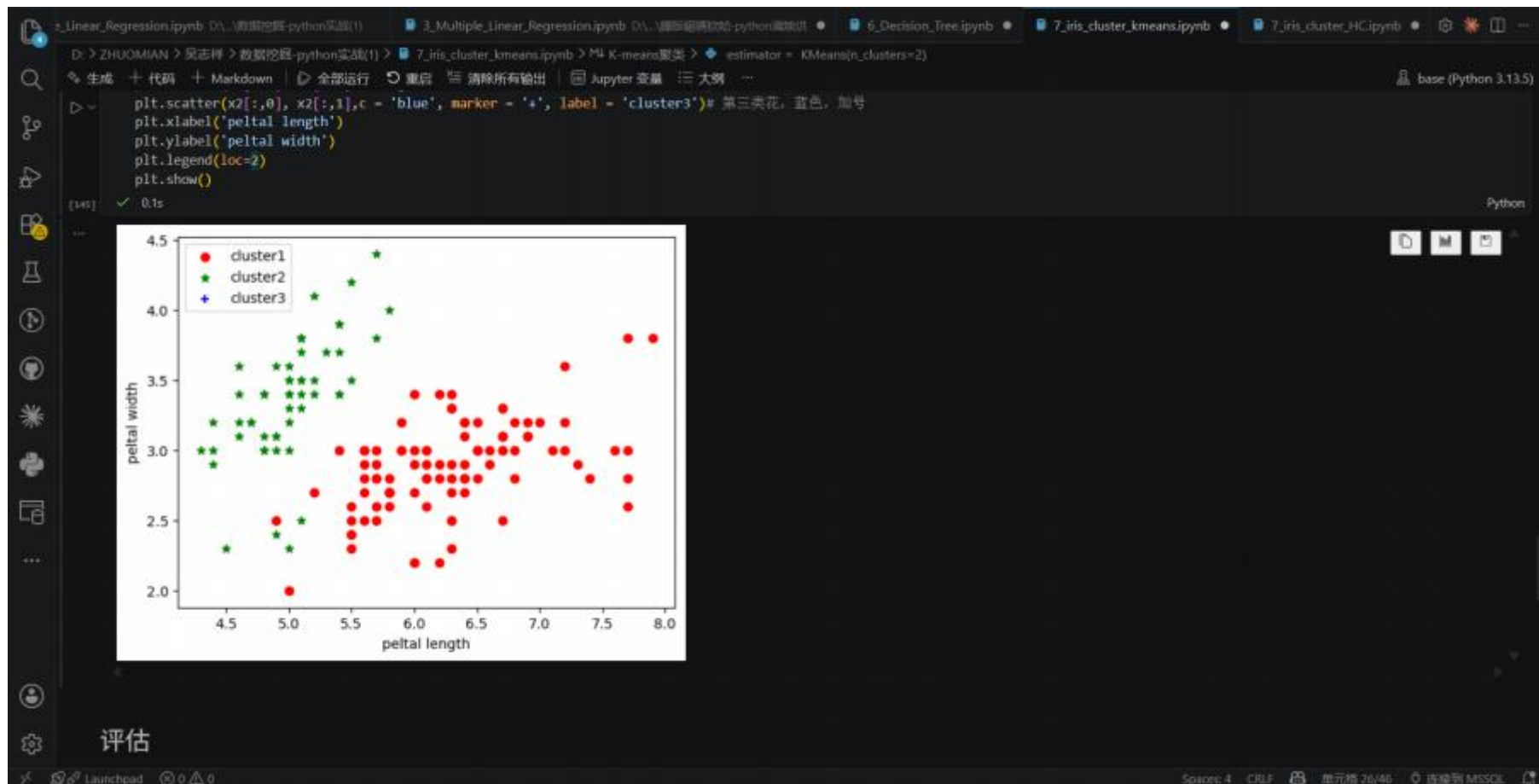
1. 用花萼的数据来聚类



2. 用所有数据来聚类



3. 把K改成2试试



4. 查看聚类中心和可视化

